Classification of Hyperspectral Images via Multitask Generative Adversarial Networks

Renlong Hang, *Member, IEEE*, Feng Zhou, Qingshan Liu, *Senior Member, IEEE*, and Pedram Ghamisi, *Senior Member, IEEE*

***Abstract*— Deep learning has shown its huge potential in the field of hyperspectral image (HSI) classification. However, most of the deep learning models heavily depend on the quantity of available training samples. In this article, we propose a multitask generative adversarial network (MTGAN) to alleviate this issue by taking advantage of the rich information from unlabeled samples. Specifically, we design a generator network to simultaneously undertake two tasks: the reconstruction task and the classification task. The former task aims at reconstructing an input hyperspectral cube, including the labeled and unlabeled ones, whereas the latter task attempts to recognize the category of the cube. Meanwhile, we construct a discriminator network to discriminate the input sample coming from the real distribution or the reconstructed one. Through an adversarial learning method, the generator network will produce real-like cubes, thus indirectly improving the discrimination and generalization ability of the classification task. More importantly, in order to fully explore the useful information from shallow layers, we adopt skip-layer connections in both reconstruction and classification tasks. The proposed MTGAN model is implemented on three standard HSIs, and the experimental results show that it is able to achieve higher performance than other state-of-the-art deep learning models.**

***Index Terms*— Convolutional neural network (CNN), generative adversarial networks (GANs), hyperspectral image (HSI) classification, multitask learning.**

I. Introduction

HYPERSPECTRAL sensors are able to capture the target scene with different numbers of wavelengths. In comparison with the traditional multispectral imaging, they can provide more detailed and rich spectral information, which will benefit a broad range of remote sensing applications, such as precision agriculture and urban development. As an essential step for these applications, hyperspectral image (HSI) classification focuses on recognizing the category of each pixel from a predefined label set. Most of the traditional classification models follow a common pipeline, which consists of a feature extraction procedure [1], [2] and a classifier design procedure [3]–[5]. However, these two procedures are often independently implemented, making the extracted features may be not the optimal ones for the designed classifier and vice versa. In contrast, deep learning models combine them into a unified framework and attempt to optimize it in a data-driven fashion, thus showing huge superiority and potential for HSI classification in recent years [6], [7].

Among various deep learning models, convolutional neural networks (CNNs) are perhaps the most popular ones because of their good performance for spectral and spectral–spatial classification, whereas autoencoders (AEs) [8], [9] or recurrent neural networks (RNNs) [10], [11] are typically used to process vector inputs. For instance, Hu *et al.* [12] fed each pixel into a 1-D CNN, which mainly consisted of 1-D convolutional operators, for spectral feature extraction and classification. Due to the loss of spatial information, the classification maps produced by 1-D CNNs often contain spurious classifications. Therefore, more and more researchers attempt to design spectral–spatial CNNs [13]–[15]. A typical method is to input a small cube around each pixel into a CNN with 2-D or 3-D convolutional operators [13]. As a common issue in deep learning models, CNNs will also be easily affected by the quantity of available training samples. Although the local connection and weight sharing mechanisms have dramatically reduced the number of training parameters in CNNs, there still exist plenty of parameters, which requires a lot of labeled samples to train. Unfortunately, the labeling of pixels in HSIs is expensive and time-consuming.

In order to address the aforementioned problem, some widely adopted strategies in the field of computer vision, such as data augmentation [16] and transfer learning [17], [18], have been used for HSI classification. For example, a random zero setting strategy was proposed in [19] for generating new samples. Li *et al.* [20] considered a pixelwise classification problem as a pixel-pair counterpart, thus indirectly increasing the number of training samples. Jiao *et al.* [21] and Cheng *et al.* [22] demonstrated that CNNs trained from natural images can be transferred to extract spatial features of HSIs. Different from them, another promising strategy is to leverage the rich information from unlabeled samples [23],

Manuscript received September 17, 2019; revised May 5, 2020 and June 12, 2020; accepted June 12, 2020. Date of publication June 25, 2020; date of current version January 21, 2021. This work was supported in part by the Natural Science Foundation of China under Grant 61825601, Grant 61532009, and Grant 61906096, and in part by the Natural Science Foundation of Jiangsu Province, China, under Grant BK20180786 and Grant 18KJB520032. *(Corresponding author: Qingshan Liu.)*

Renlong Hang, Feng Zhou, and Qingshan Liu are with the Jiangsu Key Laboratory of Big Data Analysis Technology, the School of Automation, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: renlong_hang@163.com; 13057588879@163.com; qsliu@nuist.edu.cn).

Pedram Ghamisi is with Helmholtz-Zentrum Dresden-Rossendorf (HZDR), Helmholtz Institute Freiberg for Resource Technology (HIF), D-09599 Freiberg, Germany (e-mail: p.ghamisi@gmail.com).

Color versions of one or more of the figures in this article are available online at https://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TGRS.2020.3003341

0196-2892 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

which is easy to acquire. AE is an early attempt along this direction [8]. The core idea behind AE is using unlabeled samples to realize a reconstruction task and then using labeled samples to fine-tune the network for a classification task. By taking advantage of the large numbers of unlabeled samples, AE is able to acquire robust and powerful discrimination features for HSI classification. Recently, Mou *et al.* [24] extended this framework to HSI spectral–spatial classification. Similar to AE, they applied a convolutional subnetwork together with a deconvolutional subnetwork for the reconstruction task and only the convolutional subnetwork for the classification task. In this article, we also focus on the use of unlabeled samples under the framework of generative adversarial networks (GANs) to improve the discrimination and generalization ability of CNNs.

GAN, as a kind of generative model, was first proposed in [25]. It provides a new pipeline for high-quality samples generation in an adversarial learning manner between a generator and a discriminator, thus quickly drawing great attention and becoming one of the most active research topics. In the past few years, a series of modification works based on GAN was developed. Popular models include conditional GAN [26], deep convolutional GAN [27], cycle-consistent GAN [28], least-squares GAN [29], and Wasserstein GAN [30]. Radford *et al.* [27] designed a feasible scheme for using CNNs to produce images from a random noise. Isola *et al.* [31] suggested that GANs are capable of solving various image-to-image translation problems. According to the specific applications, the inputs of generators can be any kind of images in addition to noise vectors. Inspired from them, we also use CNNs to construct our model whose input is a small cube extracted around each pixel in HSIs. Different from them, our ultimate goal is to classify each cube instead of generating new samples.

In the remote sensing community, GANs have begun to show their power in various tasks recently. For instance, conditional GAN was employed to remove the thin cloud existed in a multispectral image [32] and to simulate height image from an optical image [33]; deep convolutional GAN was applied to improve the classification results of scene images [34]. For HSI classification, there also exist some explorations [35]–[39]. Zhan *et al.* [36] proposed a 1-D GAN for HSI spectral classification. In their model, the generator uses 1-D CNN to generate hyperspectral samples from a random noise, whereas the discriminator tries to discriminate the real samples from fake (generated) ones. After training 1-D GAN, the discriminator is fine-tuned for classification with a small number of labeled samples. Since both generator and discriminator are based on 1-D CNNs, 1-D GAN cannot take advantage of spatial information of HSIs. In [37], a deconvolutional network and a 2-D CNN were designed to build the generator and the discriminator, respectively. The final two layers in the discriminator were used as an unsupervised spectral–spatial feature. For HSI classification, this model needs to train an extra classifier. Different from them, Zhu *et al.* [38] directly constructed a 3-D GAN for HSI spectral–spatial classification. However, they simplified the HSIs using three principle components, which may lose the effective spectral information for classification. Therefore, applying GANs to HSI spectral–spatial classification is still a very challenging problem.

In this article, we propose a multitask GAN (MTGAN) for HSI spectral–spatial classification. In MTGAN, the generator simultaneously undertakes two different tasks: the reconstruction task and the classification task. The reconstruction task aims at reconstructing the input HSI cube by using an encoder subnetwork and a decoder subnetwork, while the goal of the classification task is to recognize the category of the input cube via a CNN. The first convolutional layers of CNN in the classification task share the same structure of the encoder subnetwork in the reconstruction task. The discriminator of MTGAN is built by another CNN, whose inputs are the real HSI cubes or the reconstructed ones by the generator. During the training process, the discriminator tries to accurately discriminate the real HSI cubes from the reconstructed ones, whereas the generator attempts to fool the discriminator by reconstructing cubes as real as possible. After this adversarial training, the generator is capable of reconstructing real-like cubes, which indicates that the encoder subnetwork captures the intrinsic representation of the HSI cubes. Since the classification task shares the structure of the encoder subnetwork, its classification ability will also be improved. This improvement benefits from the use of unlabeled samples, which is much cheaper to acquire than labeled samples. More importantly, we adopt skip-layer connections in both reconstruction and classification tasks. In the case of reconstruction task, these skip-layer connections will bring the spatial information in the shallow layers to the deep layers, thus aiding the reconstruction of high-quality cubes. For the classification task, shallow convolutional layers will provide extra discriminative information to the deep convolutional layers, thus improving the classification performance.

Similar to the models in [8] and [24], the proposed MTGAN model also leverages the reconstruction idea to make use of unlabeled samples. Different from them, this idea is embedded into a more powerful framework named GAN, which allows maximizing the potential of the reconstruction task. In addition, the classification task is directly combined with the reconstruction task, thus making that the generator has not only the reconstruction ability but also enough discriminative ability. In comparison with the existing GAN-based HSI classification models, our proposed MTGAN model has at least two advantages. First, unlike [36] and [37], the generator in MTGAN can be directly applied to HSI classification after adversarial training, without needing to train extra classifiers, which may also cost much time. Second, MTGAN is able to sufficiently learn the spectral and spatial features simultaneously from the input HSI cube, which is essential for HSI spectral–spatial classification. However, the model in [36] can only process spectral information, whereas the model in [38] easily leads to the loss of spectral information after the preprocessing of HSIs. In summary, the main contributions of this article are as follows.

1) We propose an MTGAN for HSI spectral–spatial classification. Both the generator and the discriminator in MTGAN are based on CNNs. The generator is designed

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

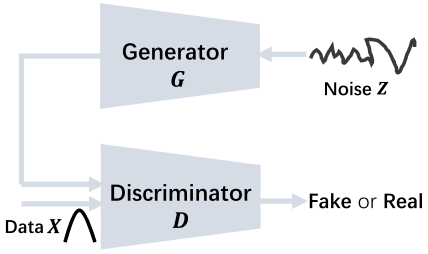


Fig. 1. Framework of GAN.

to reconstruct and classify input cubes simultaneously. Benefiting from the reconstruction process, the unlabeled samples are sufficiently exploited to improve the classification performance of CNNs.

2) We adopt skip-layer connections in both the reconstruction task and the classification task. Via these connections, the spatial information and the discriminative information in the shallow layers can be transferred to aid the reconstruction and classification tasks, respectively.
3) We test the proposed MTGAN model on three challenging HSIs: Indian Pines 2010, Houston 2013, and Houston 2018. On all of them, MTGAN is capable of achieving higher performance than the compared deep learning models, which sufficiently validates the effectiveness of it.

The following sections of this article are organized as follows. In Section II, we first review the core idea of GAN and then present the details of the proposed method. In Section III, we describe three HSIs used in our experiments and the experimental results of different models on them. Finally, the concluding remarks are summarized in Section IV.

II. Methodology

A. Review of GAN

Different from the traditional deep learning models, GAN is comprised of two networks, shown in Fig. 1. One of them is the generator G, and the other one is the discriminator D. G takes a random noise $\boldsymbol{z}$ as input and tries to generate fake data $G(\boldsymbol{z})$. The input of D is either the real data $\boldsymbol{x}$ or the generated one $G(\boldsymbol{z})$. The output of D denotes an estimated probability that the input comes from the real data. During the training process of GAN, G and D have contradictory objectives. In particular, G attempts to generate real-like data to fool D, whereas D aims at distinguishing the real data from the generated one accurately. This process can be formulated as a min-max problem

$$\min_G \max_D V(G,D) = E_{\boldsymbol{x}\sim p(\boldsymbol{x})}[\log D(\boldsymbol{x})] + E_{\boldsymbol{z}\sim p(\boldsymbol{z})}[\log(1-D(G(\boldsymbol{z})))] \quad (1)$$

where E represents the expectation and $p(\boldsymbol{x})$ and $p(\boldsymbol{z})$ denote the distributions of the real data and the noise, respectively. In practice, the solution of (1) is acquired via an alternative optimization method. First, G is fixed, whereas D is optimized by maximizing $E_{\boldsymbol{x}\sim p(\boldsymbol{x})}[\log D(\boldsymbol{x})]+E_{\boldsymbol{z}\sim p(\boldsymbol{z})}[\log(1-D(G(\boldsymbol{z})))]$.

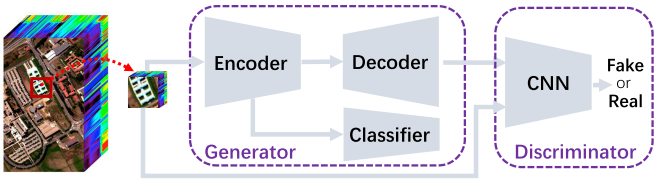


Fig. 2. Framework of the proposed MTGAN model.

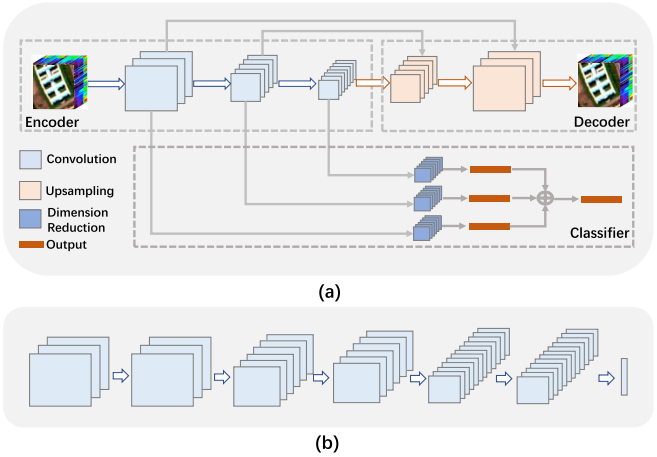


Fig. 3. Architectures of (a) generator and (b) discriminator in MTGAN.

Then, D is fixed, whereas G is optimized by minimizing $E_{\boldsymbol{z}\sim p(\boldsymbol{z})}[\log(1-D(G(\boldsymbol{z})))]$. As discussed in [25], after enough iterations, $p(\boldsymbol{z})$ will converge to $p(\boldsymbol{x})$, which indicates that G is capable of learning the true distribution of $p(\boldsymbol{x})$.

B. Architecture of MTGAN

Inspired from the powerful learning ability of GAN, our proposed MTGAN model attempts to incorporate a multitask learning framework into GAN, which can make full use of unlabeled samples to improve the classification performance of CNNs. As shown in Fig. 2, the generator of MTGAN contains three modules: the encoder module, the decoder module, and the classifier module. The input of the generator is a small cube cropped around each pixel, instead of a random noise. The encoder and decoder modules are combined together to reconstruct the input cube, whereas the encoder and the classifier modules are used to classify the input cube. These two tasks share the encoder module, making them promote from each other. The discriminator of MTGAN is constructed by a CNN, which is expected to differentiate the real cubes from the reconstructed ones. In the following, we will introduce the architectures of the generator and the discriminator in detail.

1) Encoder Module: Fig. 3(a) shows the architecture of the generator network. For a given HSI cube $\mathbf{X} \in \mathbb{R}^{w\times w\times d}$, where w and d represent the spatial size of the cube and the number of spectral bands in HSIs, respectively, the encoder module aims to learn latent representations of $\mathbf{X}$ via three 2-D convolutional layers. The kernel size of these three convolutional layers is the same (i.e., 3×3), and the number of kernels for them is 32, 64, and 128, respectively. Assume that $\mathbf{X}^{[L]}$, where $L \in \{1,2,3\}$, is the output of the $[L]$th convolutional layer,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

the computational process can be written as

$$\mathbf{X}^{[L]} = \mathrm{ReLU}(\mathbf{W}^{[L]} * \mathbf{X}^{[L-1]} + \mathbf{b}^{[L]}) \tag{2}$$

where $\mathbf{X}^{[0]} = \mathbf{X}$, $\mathbf{W}^{[L]}$ is the $[L]$th convolutional kernel, "$*$" is the convolutional operator, $\mathbf{b}^{[L]}$ is the bias, and $\mathrm{ReLU}(\cdot)$ is the widely used nonlinear activation function

$$\mathrm{ReLU}(x) = \begin{cases} x, & \text{if } x > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Note that each convolutional layer uses a padding operator to make the output spatial size unchanged. After each convolutional operator, a max-pooling operator is added to reduce the spatial size by two times.

2) Decoder Module: In contrast to the encoder module, the objective of the decoder module is to reconstruct $\mathbf{X}$ with three upsampling layers and skip-layer connections. For the first upsampling layer, $\mathbf{X}^{[3]}$ is first upsampled by two times in the spatial domain, and then, a 2-D convolutional operator is employed to it. The computation process of the convolutional operator is the same as (2). The kernel size and the number of kernels are set to 3×3 and 64, respectively. Assume that the output of the first upsampling layer is $\widehat{\mathbf{X}}^{[2]}$, and the skip-layer connection will concatenate $\widehat{\mathbf{X}}^{[2]}$ and $\mathbf{X}^{[2]}$ together, i.e., $[\widehat{\mathbf{X}}^{[2]}, \mathbf{X}^{[2]}]$. Similar to the first upsampling layer, the second layer adopts an upsampling operator and a 2-D convolutional operator (32 convolutional kernels) sequentially on $[\widehat{\mathbf{X}}^{[2]}, \mathbf{X}^{[2]}]$ to generate an output map $\widehat{\mathbf{X}}^{[1]}$. Again, $\widehat{\mathbf{X}}^{[1]}$ and $\mathbf{X}^{[1]}$ are concatenated together via a skip-layer connection and fed into the third upsampling layer, which uses d convolutional kernels to reconstruct $\mathbf{X}$ as $\widehat{\mathbf{X}}$.

3) Classifier Module: The latent representation in the encoder module can be regarded as a spectral–spatial feature of $\mathbf{X}$. It is reasonable to use this representation for the classification task. In addition, the works in [37] and [40] indicate that the representations in different convolution layers have some complementary information, which will improve the classification performance if fully explored. Motivated by them, the classifier module leverages three skip-layer connections to integrate the features from three convolutional layers for spectral–spatial classification. Specifically, for the $[L]$th convolutional layer, the classifier module first inputs $\mathbf{X}^{[L]}$ into a dimensionality reduction layer, which consists of a 1×1 convolutional operator with 256 kernels and an adaptive average-pooling operator. Thus, the dimension of the feature is reduced to 256. Then, an output layer is used to derive a classification result $\mathbf{O}^{[L]}$ via a fully connected operator. Finally, the three output results $\mathbf{O}^{[L]}$, $L \in \{1, 2, 3\}$ are combined together in an adaptive summation fashion, which can be formulated as

$$\mathbf{O} = \alpha_1 \mathbf{O}^{[1]} + \alpha_2 \mathbf{O}^{[2]} + \alpha_3 \mathbf{O}^{[3]} \tag{4}$$

where $\alpha_i, i \in \{1, 2, 3\}$ represents the fusion weight, and it is optimized together with the network parameters in an end-to-end manner.

4) Discriminator Network: Fig. 3(b) shows the structure of the discriminator network, which is comprised of six convolutional layers and an output layer. The same as the encoder module, all of the convolutional layers adopt 3×3 convolutional kernels with padding operators. The numbers of kernels from the first to the sixth convolution layer are correspondingly designed as 32, 32, 64, 64, 128, and 128. Different from the encoder module, strided convolutions, instead of max-pooling operators, are used in the second, the fourth, and the sixth layers, to learn the spatial downsampling by two times. In addition, the works in [27] found that using leaky ReLU (LReLU) achieves better results than ReLU. Therefore, we also adopt LReLU as the activation function for each convolutional layer, whose computational formula is

$$\mathrm{LReLU}(x) = \begin{cases} x, & \text{if } x > 0, \\ ax, & \text{otherwise} \end{cases} \tag{5}$$

where a is empirically set to 0.2 in this article. After the last convolution layer, an average pooling operator is applied to reduce the feature dimension to 128. The number of neurons in the output layer is 1, which indicates the probability of the input $\mathbf{X}$ or $\widehat{\mathbf{X}}$ belonging to the real cube.

C. Overall Loss Function

In addition to the network structure, the design of suitable loss functions is also essential for deep learning models. According to the objectives of the generator and the discriminator networks, we propose three different loss functions for the MTGAN model, including the reconstruction loss, the classification loss, and the GAN loss. These loss functions will be discussed in the following.

1) Reconstruction Loss: An intuitive idea for the reconstruction task is to minimize the pixelwise loss between the input cube and the reconstructed one [29], which can be modeled as

$$\mathcal{L}_{\mathrm{rec}} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{X}_i - G_{\theta_1}(\mathbf{X}_i)\|^2 \tag{6}$$

where N denotes the total number of samples, θ_1 denotes the parameters existed in the encoder module and the decoder module, and $\mathbf{X}_i$ and $G_{\theta_1}(\mathbf{X}_i)$ represent the ith input cube and the correspondingly reconstructed one, respectively. The reconstruction loss plays two roles in this article. First, it guides the generator to realize a reconstruction task. Second, it helps improve the discrimination and generalization abilities of the classifier module.

2) Classification Loss: As a widely adopted method, cross-entropy loss is used to construct the classification loss function, which can be written as

$$\mathcal{L}_{\mathrm{cls}} = \frac{1}{M} \sum_{i=1}^{M} [y_i \log(G_{\theta_2}(\mathbf{X}_i)) + (1 - y_i) \log(1 - G_{\theta_2}(\mathbf{X}_i))] \tag{7}$$

where M denotes the number of labeled cubes, y_i denotes the class label of the ith input cube, and θ_2 represents the parameters contained in the encoder module and the classifier module. Note that θ_1 and θ_2 share the parameters in the encoder module, making the reconstruction task and the classification task promote from each other.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

3) GAN Loss: It is well known that the loss function in (6) easily leads to the loss of high-frequency information, resulting in blurred reconstruction results [31], [41]. In order to generate more realistic results, a Wasserstein distance-based GAN loss is introduced, which can be defined as

$$\mathcal{L}_{\text{gan}} = \frac{1}{N}\sum_{i=1}^{N} D_\beta(\mathbf{X}_i) - D_\beta(G_{\theta_1}(\mathbf{X}_i)) \tag{8}$$

where β denotes the network parameters in the discriminator.

4) Objective Function: The final objective function $\mathcal{L}$ of the proposed MTGAN model is a combination of the reconstruction loss in (6), the classification loss in (7), and the GAN loss in (8)

$$\mathcal{L} = \arg\min_{(\theta_1,\theta_2)} \max_{\beta} \quad \mathcal{L}_{\text{cls}} + \lambda_1 \mathcal{L}_{\text{rec}} + \lambda_2 \mathcal{L}_{\text{gan}} \tag{9}$$

where λ_1 and λ_2 are regularization parameters used to trade off different loss terms.

The same as the original GAN work in [25], we also adopt an alternative optimization method to optimize (9). Thus, (9) can be decomposed into two subproblems

$$\arg\max_{\beta} \frac{1}{N}\sum_{i=1}^{N} D_\beta(\mathbf{X}_i) - D_\beta(G_{\theta_1}(\mathbf{X}_i)) \tag{10}$$

and

$$\arg\min_{(\theta_1,\theta_2)} \mathcal{L}_{\text{cls}} + \lambda_1 \mathcal{L}_{rec} - \lambda_2 \frac{1}{N}\sum_{i=1}^{N} D_\beta(G_{\theta_1}(\mathbf{X}_i)). \tag{11}$$

After acquiring the optimal solutions of θ_1, θ_2, and β at the training phase, we can use the encoder module and the classifier module to classify any input cubes for HSI spectral–spatial classification.

III. Experiments

In this section, we conduct experiments on three challenging HSI data sets to test the performance of the proposed MTGAN model. Different evaluation indicators are employed, including each-class accuracies, average accuracy (AA), overall accuracy (OA), and Kappa coefficient.

A. Data Description

The first HSI data set is Indian Pines 2010. It consists of 360 spectral bands, which is a very high-resolution HSI. The spatial size of the data is 445 × 750. There exist 16 different classes of land covers to recognize. Fig. 4 shows the RGB-composite image, the training map, and the test map of this data. The detailed sample distribution is given in Table I. The second HSI data set is Houston 2013 [42], which contains 349 × 1905 pixels and 144 spectral bands. As shown in Fig. 5, there are 15 land-cover classes to discriminate. Table II reports the available sample distribution in each class for training and testing. The third HSI data set is Houston 2018 [43], [44]. In this article, we use the training portion of the whole data set, which was distributed by the Image Analysis and Data Fusion Technical Committee of the IEEE Geoscience and Remote Sensing Society (GRSS) and the University of Houston for the 2018 data fusion contest. In comparison with the Houston 2013 data, it has a larger spatial size (i.e., 601 × 2384) but less spectral bands (i.e., 48). As shown in Fig. 6, the number of land-cover classes is 20, making it more difficult than the Houston 2013 data. Table III demonstrates the number of training samples and test samples, as well as the total number of samples in each class.

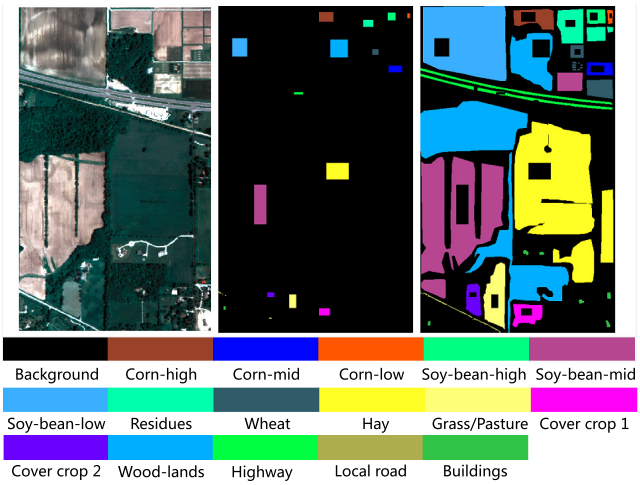


Fig. 4. Visualization of the Indian Pines 2010 data set. (From left to right) RGB-composite image, the training map, and the test map, respectively.

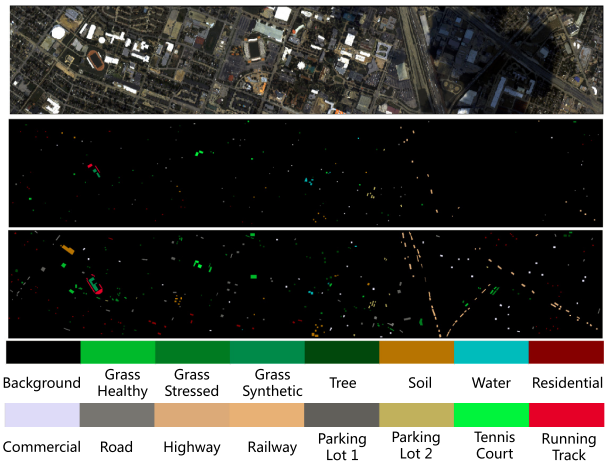


Fig. 5. Visualization of the Houston 2013 data set. (From top to bottom) RGB-composite image, the training map, and the test map, respectively.

B. Experimental Setup

To test the superiority of the proposed MTGAN model, we compare it with several state-of-the-art deep learning models, including AE, RNN, CNN, convolutional AE (CAE), HS-GAN in [36], and WGAN in [37]. Besides, we also use the original spectral signature combined with the random forest (RF) classifier as a baseline. In the following, we will give some descriptions of these comparison methods.

1) *RF:* The original spectral signature is fed into an RF classifier. The number of trees in RF is 200 [44].
2) *AE:* It is comprised of three hidden layers. The neuron numbers in each hidden layer are sequentially set to 32, 64, and 128.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I

SAMPLE DISTRIBUTIONS ON THE INDIAN PINES 2010 DATA SET: THE TRAINING SAMPLES, THE TEST SAMPLES, AND THE TOTAL NUMBER OF SAMPLES IN EACH CLASS

Class No.	Class Name	Training	Test	Total
1	Corn-high	726	2661	3387
2	Corn-mid	465	1275	1740
3	Corn-low	66	290	356
4	Soy-bean-high	324	1041	1365
5	Soy-bean-mid	2548	35317	37865
6	Soy-bean-low	1428	27782	29210
7	Residues	368	5427	5795
8	Wheat	182	3205	3387
9	Hay	1938	48107	50045
10	Grass/Pasture	496	5048	5544
11	Cover crop 1	400	2346	2746
12	Cover crop 2	176	1988	2164
13	Woodlands	1640	46919	48559
14	Highway	105	4758	4863
15	Local road	52	450	502
16	Buildings	40	506	546
-	Total	10954	187120	198074

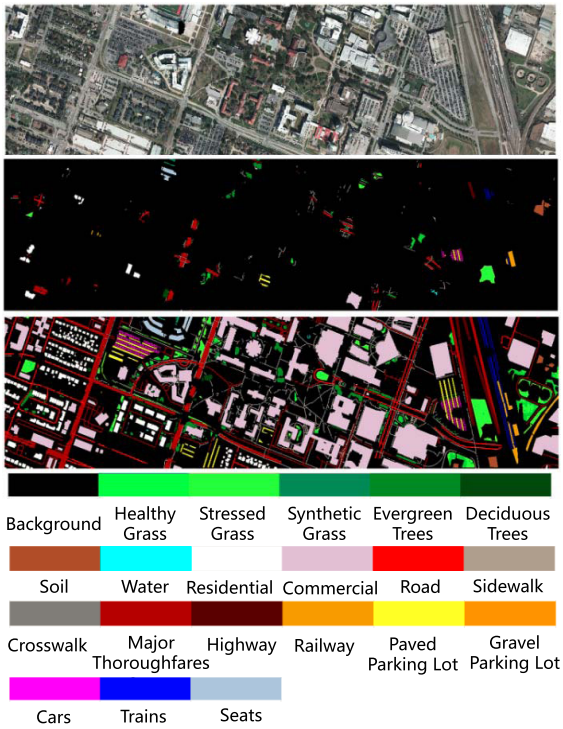


Fig. 6. Visualization of the Houston 2018 data set. (From top to bottom) RGB-composite image, the training map, and the test map, respectively.

TABLE II

SAMPLE DISTRIBUTIONS ON THE HOUSTON 2013 DATA SET: THE TRAINING SAMPLES, THE TEST SAMPLES, AND THE TOTAL NUMBER OF SAMPLES IN EACH CLASS

Class No.	Class Name	Training	Test	Total
1	Grass Healthy	198	1053	1251
2	Grass Stressed	190	1064	1254
3	Grass Synthetic	192	505	697
4	Tree	188	1056	1244
5	Soil	186	1056	1242
6	Water	182	143	325
7	Residential	196	1072	1268
8	Commercial	191	1053	1244
9	Road	193	1059	1252
10	Highway	191	1036	1227
11	Railway	181	1054	1235
12	Parking Lot 1	192	1041	1233
13	Parking Lot 2	184	285	469
14	Tennis Court	181	247	428
15	Running Track	187	473	660
-	Total	2832	12197	15029

TABLE III

SAMPLE DISTRIBUTIONS ON THE HOUSTON 2018 DATA SET: THE TRAINING SAMPLES, THE TEST SAMPLES, AND THE TOTAL NUMBER OF SAMPLES IN EACH CLASS

Class No.	Class Name	Training	Test	Total
1	Healthy grass	1458	8341	9799
2	Stressed grass	4316	28186	32502
3	Synthetic grass	331	353	684
4	Evergreen Trees	2005	11583	13588
5	Deciduous Trees	676	4372	5048
6	Soil	1757	2759	4516
7	Water	147	119	266
8	Residential	3809	35953	39762
9	Commercial	2789	220895	223684
10	Road	3188	42622	45810
11	Sidewalk	2699	31303	34002
12	Crosswalk	225	1291	1516
13	Major Thoroughfares	5193	41165	46358
14	Highway	700	9149	9849
15	Railway	1224	5713	6937
16	Paved Parking Lot	1179	10296	11475
17	Gravel Parking Lot	127	22	149
18	Cars	848	5730	6578
19	Trains	493	4872	5365
20	Seats	1313	5511	6824
-	Total	34477	470235	504712

3) *RNN:* The original spectral signature is regarded as a sequence and fed into RNN for classification. The number of recurrent layers is 2, and each recurrent layer contains 128 neurons.
4) *CNN:* It consists of three convolutional layers with 3 kernels and an output layer. The number of kernels in each convolutional layer is sequentially set to 32, 64, and 128.
5) *CAE:* Similar to the combination of encoder and decoder modules in MTGAN, CAE consists of three convolutional layers and three deconvolutional layers without skip-layer connections. At the classification phase, these three convolutional layers are followed by another convolutional layer with 256 3×3 kernels and an output layer.
6) *HS-GAN:* The same as the work in [36], GAN is first used to generate a spectral signature, and then, the discriminator is fine-tuned for HSI classification.
7) *WGAN:* The same as [37], WGAN takes a random noise as input and tries to generate a hyperspectral cube. After that, the convolutional features in the discriminator are used for HSI spectral–spatial classification.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

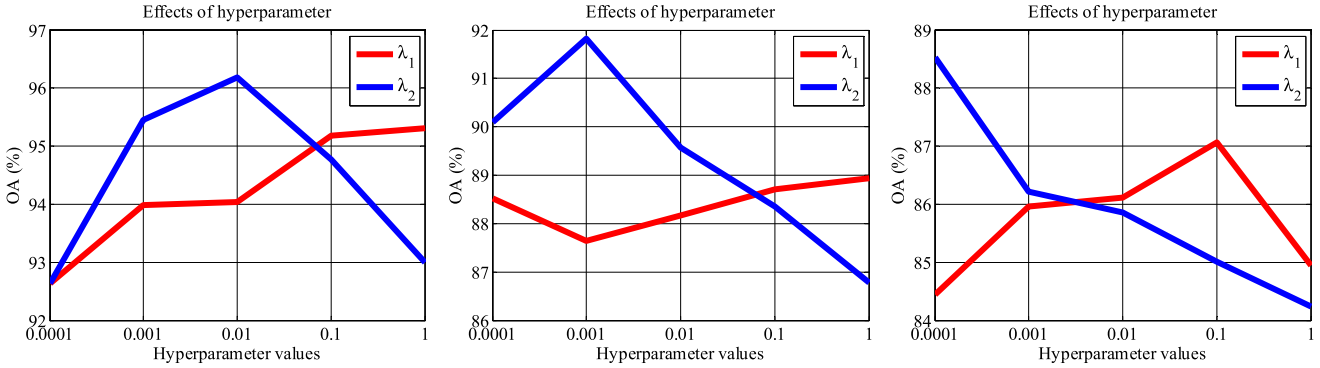


Fig. 7. Effects of λ_1 and λ_2 on classification performance. (From left to right) Indian Pines 2010 data, the Houston 2013 data, and the Houston 2018 data.

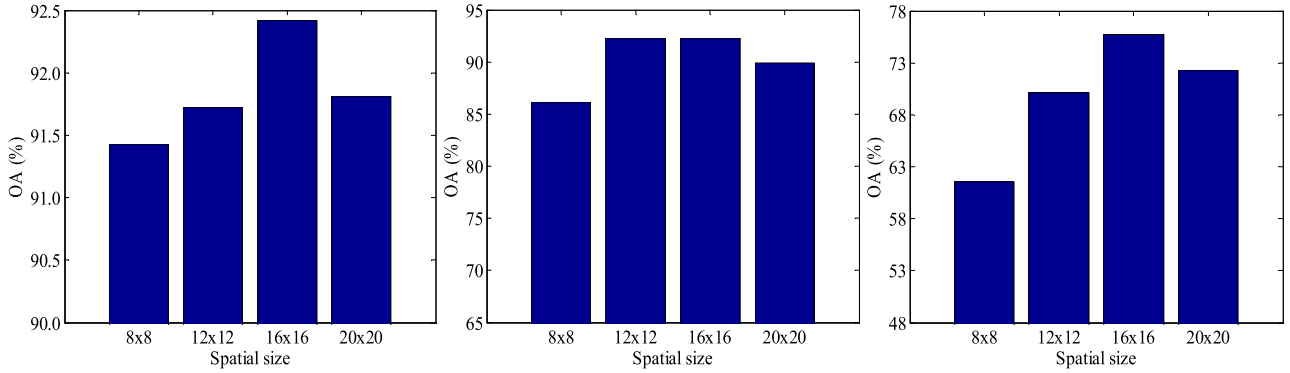


Fig. 8. Effects of cube size on classification performance. (From left to right) Indian Pines 2010 data, the Houston 2013 data, and the Houston 2018 data.

For all of the deep learning-related models, we use the PyTorch framework to implement the experiments. The optimizer in the deep learning-related models is Adam with default parameters. For the CNN, CAE, and MTGAN models, the input cube size is empirically set to $16 \times 16 \times d$, where d represents the number of spectral bands in the original HSIs. Since deep learning models may be affected by different initialization values, we repeat them ten times and report the averaged results to alleviate this issue.

C. Analysis on the MTGAN Model

1) Effects of Hyperparameters λ_1 and λ_2: There are two regularization parameters in the objective function of MTGAN. To test their effects on the classification performance, we use a fivefold cross-validation method on the training set, and the parameter values are chosen from a candidate set $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$. Fig. 7 shows the averaged OAs acquired by MTGAN with different λ_1 (red curves) or λ_2 (blue curves) values. When $\lambda_1 = 0.1$, MTGAN can achieve the highest OA on the Houston 2018 data or the values close to the highest ones on the Indian Pines 2010 and the Houston 2013 data. Therefore, it is reasonable to choose 0.1 for λ_1. Compared to λ_1, OA varies more significantly in terms of λ_2. Specifically, on the Indian Pines 2010 data, OA will first increase and reach the highest value when $\lambda_2 = 0.01$ but will quickly decrease when $\lambda_2 > 0.01$. Thus, λ_2 is set to 0.01 on the Indian Pines 2010 data. Similarly, it is obvious to find that $\lambda_2 = 0.001$ and $\lambda_2 = 0.0001$ are the best choices for the Houston 2013 and the Houston 2018 data, respectively.

2) Effects of Input Cube Sizes: Besides the regularization parameters λ_1 and λ_2, the spatial size of input cubes will also affect the performance of MTGAN. Larger cubes may contain unrelated or even corrupted information, which will decrease the classification performance. On the contrary, smaller cubes may lose useful discriminant information, which will also degrade the performance. Therefore, it is important to select a suitable cube size. Fig. 8 shows the effects of four different cube sizes (i.e., 8×8, 12×12, 16×16, and 20×20) on the classification performance of MTGAN. It can be observed that MTGAN is able to obtain the highest OAs when the cube size is 16×16 on all of the three data sets.

3) Effects of Skip-Layer Connections: Both the reconstruction task and the classification task contain skip-layer connections in MTGAN. To validate the effectiveness of them, we compare MTGAN with two baseline models. The first one is MTGAN-RC, which removes the skip-layer connections in the reconstruction and classification tasks. The second one is MTGAN-C, which only removes the skip-layer connections in the classification task. As shown in Fig. 9, MTGAN-C can achieve higher performance than MTGAN-RC in terms of OA, AA, and Kappa, whereas MTGAN further improves the performance in comparison with MTGAN-C. These observations indicate the effectiveness of skip-layer connections.

4) Effects of the Adversarial Network: Adversarial loss is an essential component of MTGAN. For testing its effects on the classification performance, we first set λ_2 to 0 and choose the optimal λ_1 value (i.e., 0.1) to acquire a baseline result without using GAN. Then, we change λ_2 to its optimal

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

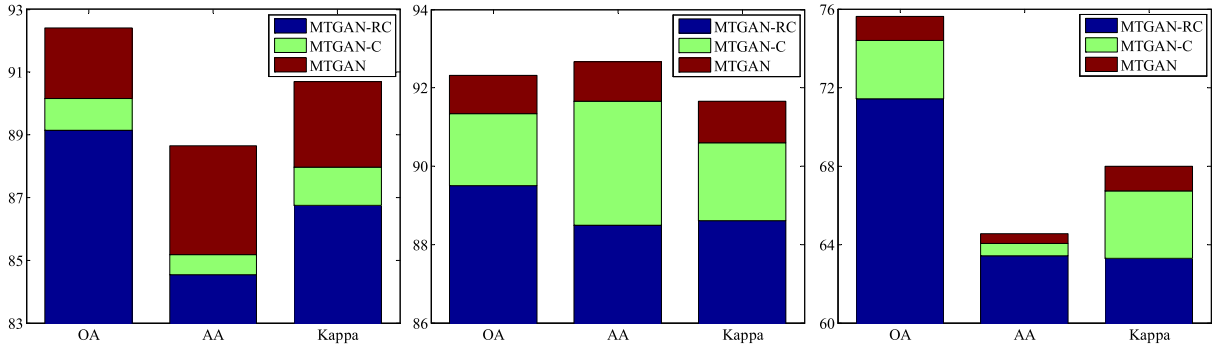


Fig. 9. Effects of skip-layer connections on classification performance. (From left to right) Indian Pines 2010 data, the Houston 2013 data, and the Houston 2018 data.

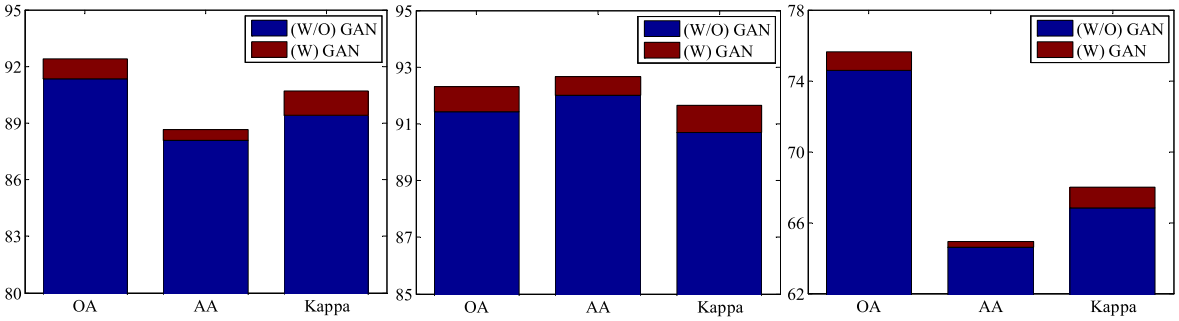


Fig. 10. Effects of GAN on classification performance. (From left to right) Indian Pines 2010 data, the Houston 2013 data, and the Houston 2018 data.

TABLE IV
WEIGHT VALUES ON DIFFERENT DATA SETS

Data	α_1	α_2	α_3
Indian Pines 2010	0.33	0.33	0.34
Houston 2013	0.31	0.33	0.36
Houston 2018	0.28	0.30	0.42

value on different data to achieve another result with the help of GAN. Fig. 10 compares their performance in terms of OA, AA, and Kappa. In this figure, blue bars are the results without GAN, whereas red bars represent the performance improvements after adding GAN. Based on these results, it is able to conclude that GAN indeed has a positive contribution to the classification task.

5) Effects of Fusion Weights: As shown in (4), the classification result is affected by the fusion weight $\alpha_i, i \in \{1, 2, 3\}$. Here, we report the optimized weight values on different data sets in Table IV. From this table, we can observe that, for each data, α_3 has the largest value because deep convolutional layers often contain more semantic information than the shallow ones. More importantly, the optimized weight values vary on different data sets. For instance, α_1, α_2, and α_3 equal to 0.31, 0.33, and 0.36, respectively, on the Houston 2013 data, while their values are 0.28, 0.30, and 0.42, on the Houston 2018 data. This indicates that it is necessary to automatically learn the weight values from data.

D. Performance Comparisons

1) Results on the Indian Pines 2010 Data: Table V reports the detailed classification results achieved by different models. In this table, the first 16 rows denote each-class accuracies, whereas the next three ones correspond to OA, AA, and Kappa of the whole data. For each row, the bold fonts indicate the best performance. Based on these results, several conclusions can be derived. First, the spectral–spatial deep models (i.e., CNN, CAE, WGAN, and MTGAN) achieve higher performance than the spectral ones (i.e., AE, RNN, and HS-GAN) in terms of OA and Kappa, due to the joint use of spectral and spatial information. In particular, among the spectral models, AE acquires the highest OA 88.36% and the largest Kappa value 0.858. For the spectral–spatial models, CAE obtains the lowest OA 88.73% and the smallest Kappa value 0.863, which are already larger than the corresponding ones achieved by AE. Second, for the reconstruction-based models, CAE is slightly inferior to CNN, although it attempts to make use of unlabeled samples. This is caused by two reasons. The first one is that the reconstruction ability of CAE is not good enough to generate powerful features. The other one is that the convolutional features in CAE are not fully exploited. After solving these two issues, our proposed model MTGAN is capable of improving OA, AA, and Kappa by about 4% compared to CAE. Third, in terms of GAN-related models, WGAN increases the classification accuracies of HS-GAN in most classes, leading to higher OA, AA, and Kappa values. However, both of them focus on the task of sample generation, instead of the classification task, so it is difficult for them to ensure the discriminative ability of learned features. In contrast to them, our MTGAN model directly deals with the classification task, thus achieving the best performance in terms of OA, AA, and Kappa. Specifically, the OA, AA, and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CLASSIFICATION RESULTS (%) ACHIEVED BY DIFFERENT MODELS ON THE INDIAN PINES 2010 DATA

Class	RF	AE	RNN	CNN	CAE	HS-GAN	WGAN	MTGAN
Corn-high	92.60	93.27	88.29	92.75	89.67	89.28	93.43	**99.31**
Corn-mid	87.69	95.73	91.78	95.44	99.18	97.52	**99.97**	99.90
Corn-low	88.62	96.83	94.05	98.90	99.03	96.97	97.59	**100**
Soy-bean-high	68.88	75.08	76.21	**88.40**	83.36	74.49	83.42	81.20
Soy-bean-mid	80.58	84.88	84.74	86.92	84.52	84.01	86.61	**91.25**
Soy-bean-low	81.72	90.13	92.04	92.68	90.18	86.43	95.86	**97.27**
Residues	41.70	62.65	57.95	65.07	60.81	65.16	68.42	**74.63**
Wheat	25.30	53.49	28.40	**87.76**	75.93	45.02	29.73	86.57
Hay	65.45	87.32	85.33	83.02	85.10	84.81	88.24	**88.83**
Grass/Pasture	82.29	90.15	80.96	83.68	83.76	87.15	82.04	**94.56**
Cover crop 1	66.58	81.21	74.14	74.40	76.50	77.75	79.19	**92.53**
Cover crop 2	99.85	99.98	97.65	99.45	98.58	99.76	99.34	**100**
Woodlands	94.68	96.21	94.27	99.25	98.83	95.60	97.94	**99.72**
Highway	87.83	90.94	90.30	99.84	99.99	91.11	98.36	**100**
Local road	93.33	93.07	81.19	99.47	99.78	91.78	98.62	**99.91**
Buildings	**37.35**	20.53	20.60	9.80	17.87	26.92	9.96	12.75
OA	78.66	88.36	86.55	89.45	88.73	86.62	89.97	**92.42**
AA	74.65	81.97	77.37	84.80	83.94	80.86	81.79	**88.65**
Kappa	73.90	85.80	83.55	87.16	86.27	83.73	87.72	**90.70**
Training(s)	-	425.15	345.36	**240.85**	555.03	3592.35	441.87	558.48
Test(s)	-	**0.062**	6.81	9.75	9.82	2.61	8.24	13.38

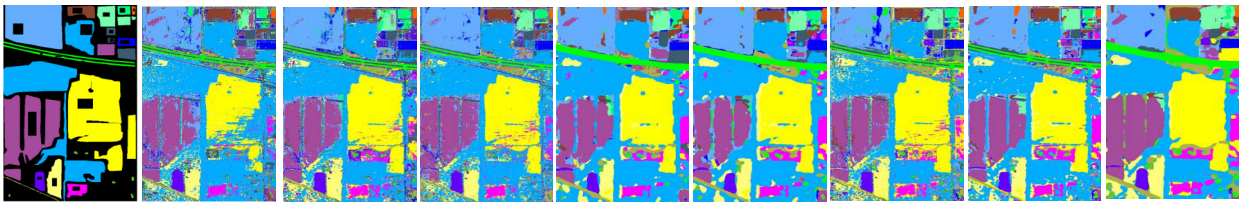


Fig. 11. Classification maps of different models on the Indian Pines 2010 data. (a) RF. (b) AE. (c) RNN. (d) CNN. (e) CAE. (f) HS-GAN. (g) WGAN. (h) MTGAN.

Kappa values are increased from 89.97% to 92.42%, 81.79% to 88.65%, and 0.877 to 0.907, respectively, when compared with WGAN. Last but not least, for each-class accuracies, MTGAN can obtain the highest values for 13 out of the total 16 classes, which also certifies the effectiveness of our proposed model.

2) Results on the Houston 2013 and 2018 Data: Tables VI and VII list the classification results of different models on the Houston 2013 and Houston 2018 data, respectively. Most of the conclusions on the Indian Pines 2010 data can be observed on these two data. For instance, the spectral–spatial models are better than the spectral models in terms of OA, AA, and Kappa. As a strong baseline, CNN is able to achieve satisfactory performance on these two data. In comparison with it, our proposed model MTGAN improves OA and Kappa values by about 3% because of the use of unlabeled samples. For the three GAN-related models, our MTGAN model significantly increases OA, AA, and Kappa values, due to the direct processing of the classification task. All the aforementioned conclusions can validate the effectiveness of our proposed model.

E. Classification Maps

In addition to the quantitative results in Tables V–VII, Figs. 11–13 also qualitatively demonstrate the classification maps of different models on the Indian Pines 2010, the Houston 2013, and the Houston 2018 data, respectively. In these figures, different colors correspond to different land-cover classes. It is obvious that the classification maps, generated by CNN, CAE, WGAN, and MTGAN, exist fewer numbers of outliers than the other ones. By comparing with the ground-truth maps, one can observe that MTGAN is able to acquire more reasonable maps than the other models, which indicates the effectiveness of the proposed model. However, similar to the other spectral-spatial models (i.e., CNN, CAE, and WGAN), the classification maps of MTGAN seem to be a little oversmoothed. The inputs of these models are cubes around each pixel, and thus, the cubes of boundary pixels between two different objects will be dominated by the information from the larger object. When inputting them into the trained network, they will be easily classified as the classes that the larger objects belong to, leading to the oversmoothed results.

F. Computation Time

The last two rows in Tables V–VII record the computation time of different models on three data sets, including the training time and the test time. Since RF is implemented on CPU, instead of GPU, we do not record its computation time. As expected, the training stage costs much more time than

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
CLASSIFICATION RESULTS (%) ACHIEVED BY DIFFERENT MODELS ON THE HOUSTON 2013 DATA

Class	RF	AE	RNN	CNN	CAE	HS-GAN	WGAN	MTGAN
Grass Healthy	82.62	82.68	82.41	82.40	81.52	**84.48**	83.06	79.83
Grass Stressed	83.18	96.33	93.87	**99.57**	97.22	94.08	99.25	98.54
Grass Synthetic	97.82	97.43	98.42	86.18	89.62	**99.96**	85.23	91.21
Tree	91.38	97.20	96.78	98.39	98.16	98.58	**98.81**	95.27
Soil	96.59	94.43	94.39	**99.58**	97.05	96.84	99.55	97.81
Water	**99.30**	97.20	96.64	83.85	83.08	95.94	91.19	98.25
Residential	74.63	78.81	80.11	88.80	**93.77**	87.59	86.96	91.21
Commercial	33.05	48.85	49.90	80.16	76.32	66.88	74.89	**86.13**
Road	67.71	67.54	72.62	81.35	84.97	74.01	84.10	**91.07**
Highway	42.95	63.51	70.25	78.78	92.24	61.47	66.64	**95.33**
Railway	70.11	77.53	82.75	86.92	89.49	82.45	85.43	**94.35**
Parking Lot 1	54.85	57.21	68.55	87.65	89.72	72.43	77.43	**94.04**
Parking Lot 2	61.40	39.23	61.54	82.84	85.40	58.11	**88.14**	86.60
Tennis Court	98.38	96.36	97.65	90.53	92.96	98.87	93.68	**100**
Running Track	**97.89**	97.63	97.00	84.14	71.37	97.12	86.60	90.19
OA	72.78	77.95	80.86	88.00	89.18	83.24	85.98	**92.32**
AA	76.79	79.46	82.86	87.41	88.19	84.59	86.73	**92.66**
Kappa	70.76	76.09	79.28	86.96	88.25	81.81	84.77	**91.66**
Training(s)	-	37.48	46.82	**21.22**	57.49	132.84	92.04	105.20
Test(s)	-	**0.0026**	0.19	0.39	0.40	0.13	0.37	0.49

TABLE VII
CLASSIFICATION RESULTS (%) ACHIEVED BY DIFFERENT MODELS ON THE HOUSTON 2018 DATA

Class	RF	AE	RNN	CNN	CAE	HS-GAN	WGAN	MTGAN
Healthy grass	30.88	79.40	57.02	75.16	34.61	77.24	**80.68**	50.46
Stressed grass	76.03	78.93	69.75	81.73	**89.41**	85.10	84.04	87.40
Synthetic grass	**100**	**100**	99.72	77.39	81.30	**100**	93.37	98.47
Evergreen Trees	91.34	92.21	86.13	94.44	94.85	92.36	**96.04**	95.25
Deciduous Trees	41.19	49.82	40.40	43.30	**55.15**	45.24	53.36	50.44
Soil	25.70	25.85	25.37	30.50	31.32	26.26	26.75	**32.35**
Water	30.25	30.25	30.25	29.08	28.40	**39.83**	36.64	17.98
Residential	76.57	72.16	73.56	85.38	**89.97**	76.33	84.72	86.83
Commercial	38.49	63.02	41.86	79.70	79.08	46.80	76.05	**84.45**
Road	36.03	38.19	34.65	**59.02**	56.63	44.94	44.17	45.17
Sidewalk	41.62	41.43	46.99	54.56	58.17	50.54	62.13	**64.27**
Crosswalk	1.32	1.52	**6.97**	5.11	4.29	5.58	5.82	6.27
Major Thoroughfares	45.25	45.23	47.89	51.48	52.18	47.73	54.59	**66.00**
Highway	30.19	37.89	33.09	52.89	**60.60**	32.52	35.36	43.03
Railway	63.03	51.97	52.89	62.77	61.75	**73.68**	54.57	58.26
Paved Parking Lot	64.12	74.57	70.86	**84.98**	78.81	64.08	69.72	83.67
Gravel Parking Lot	**100**	**100**	**100**	95.45	80.91	**100**	80.00	96.36
Cars	49.83	53.46	60.80	63.65	69.54	68.31	55.31	**78.62**
Trains	52.65	65.69	65.45	91.02	**93.07**	66.77	73.83	88.88
Seats	44.44	53.88	48.01	62.46	**58.40**	48.82	62.12	56.44
OA	46.34	59.38	48.51	72.78	72.84	53.89	69.75	**75.62**
AA	51.95	57.77	54.58	64.00	62.92	59.61	61.46	**64.53**
Kappa	37.32	49.48	39.36	64.74	64.85	45.02	60.73	**67.99**
Training(s)	-	711.04	**119.32**	145.96	366.94	2073.26	839.37	888.27
Test(s)	-	**0.061**	2.31	5.86	5.88	2.93	10.55	10.64

the test stage for each model. In terms of the training time, CNN takes less time than the other models on the Indian Pines 2010 and Houston 2013 data, whereas RNN takes less time on the Houston 2018 data. However, both of them costs much more test time than AE. For the GAN-related models (i.e., HS-GAN, WGAN, and MTGAN), HS-GAN requires less time to process the test set because it only needs to deal with the spectral information, but its classification performance is lower than WGAN and MTGAN. In comparison with HS-GAN, MTGAN attempts to process the spectral and spatial

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

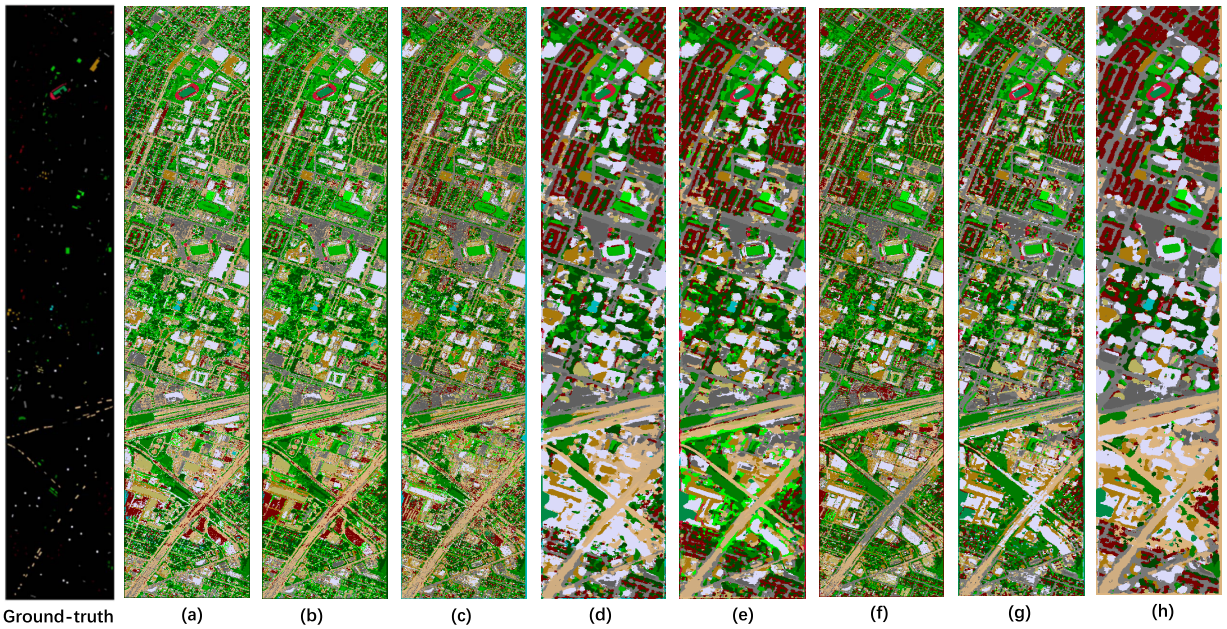


Fig. 12. Classification maps of different models on the Houston 2013 data. (a) RF. (b) AE. (c) RNN. (d) CNN. (e) CAE. (f) HS-GAN. (g) WGAN. (h) MTGAN.

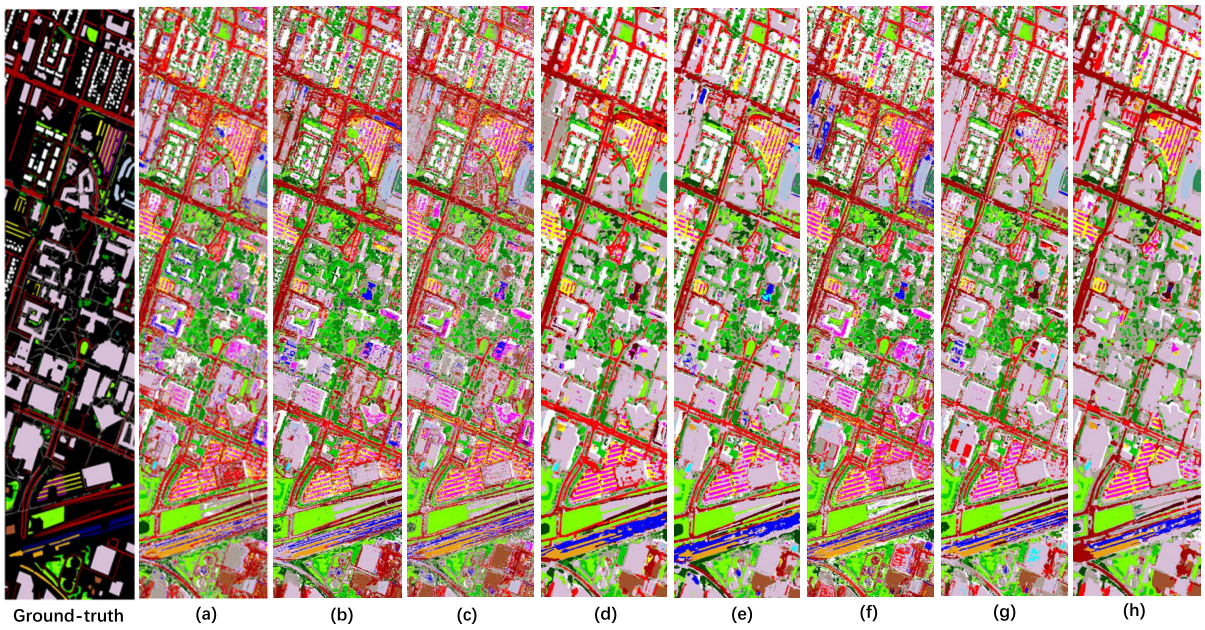


Fig. 13. Classification maps of different models on the Houston 2018 data. (a) RF. (b) AE. (c) RNN. (d) CNN. (e) CAE. (f) HS-GAN. (g) WGAN. (h) MTGAN.

information simultaneously, thus requiring more test time. However, it takes less training time than HS-GAN. More importantly, the classification performance in terms of OA, AA, and Kappa has been significantly improved by MTGAN.

G. Performance With Respect to the Number of Training Samples

In this section, we evaluate the effects of different numbers of training samples on the classification performance. Specifically, we randomly choose some percentages of samples from the original training samples to train models and use the original test samples to evaluate their performance. Fig. 14 demonstrates the OAs achieved by three different models, including MTGAN, WGAN, and CAE, when changing the training percentage from 20% to 100%. From this figure, it can be observed that as the number of training samples increases, the performance of all the three models improves. More importantly, given the same number of training samples, our proposed model MTGAN always outperforms the other two models by a large margin.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

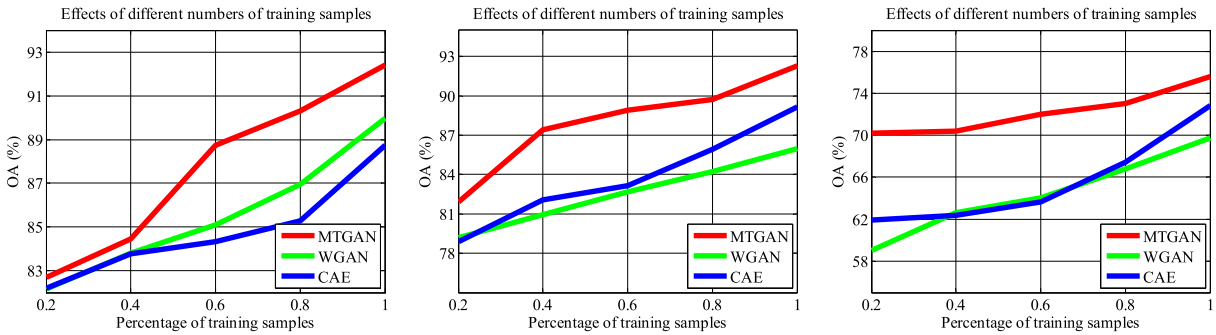


Fig. 14. Classification accuracies with respect to the number of training samples on three data. (From left to right) Indian Pines 2010 data, the Houston 2013 data, and the Houston 2018 data.

IV. Conclusion

In this article, we proposed an MTGAN for HSI classification. Different from the other GAN-based models, the proposed MTGAN model directly addresses the classification task, with the help of the reconstruction task, which can make full use of unlabeled samples. To improve the reconstruction performance, adversarial learning and skip-layer connections were adopted in MTGAN. For the classification task, skip-layer connections were also employed to take advantage of the complementary information from different convolution layers. In order to test the performance of the proposed model, we conducted experiments on three different HSI data and compared it with several deep models, including reconstruction-, CNN-, and GAN-related models. Based on the quantitative and qualitative comparisons, we are able to certify the effectiveness of MTGAN. In addition, we also attempted to make a comprehensive analysis on the proposed MTGAN model. Specifically, we tested the effects of different hyperparameters on the classification performance, including two regularization parameters and the spatial size of input cubes, and evaluated the contributions of skip-layer connections embedded in the reconstruction and classification tasks, as well as the contributions of the adversarial loss.

Acknowledgment

The author would like to thank Prof. Melba Crawford for providing the Indian Pines 2010 Data and the National Center for Airborne Laser Mapping, the Hyperspectral Image Analysis Laboratory at the University of Houston, and the IEEE GRSS Image Analysis and Data Fusion Technical Committee for providing the Houston data sets.

References

[1] R. Hang, Q. Liu, H. Song, and Y. Sun, "Matrix-based discriminant subspace ensemble for hyperspectral image spatial–spectral feature fusion," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 2, pp. 783–794, Feb. 2016.

[2] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "CoSpace: Common subspace learning from hyperspectral–multispectral correspondences," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4349–4359, Jul. 2019.

[3] P. Ghamisi, J. Plaza, Y. Chen, J. Li, and A. J. Plaza, "Advanced spectral classifiers for hyperspectral images: A review," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 1, pp. 8–32, Mar. 2017.

[4] L. He, J. Li, C. Liu, and S. Li, "Recent advances on spectral–spatial hyperspectral image classification: An overview and new guidelines," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1579–1597, Mar. 2018.

[5] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "An augmented linear mixing model to address spectral variability for hyperspectral unmixing," *IEEE Trans. Image Process.*, vol. 28, no. 4, pp. 1923–1938, Apr. 2019.

[6] R. Hang, Z. Li, P. Ghamisi, D. Hong, G. Xia, and Q. Liu, "Classification of hyperspectral and LiDAR data using coupled CNNs," *IEEE Trans. Geosci. Remote Sens.*, early access, Feb. 6, 2020, doi: 10.1109/TGRS.2020.2969024.

[7] S. Li, W. Song, L. Fang, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Deep learning for hyperspectral image classification: An overview," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6690–6709, Sep. 2019.

[8] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[9] S. Zhou, Z. Xue, and P. Du, "Semisupervised stacked autoencoder with cotraining for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 6, pp. 3813–3826, Jun. 2019.

[10] F. Zhou, R. Hang, Q. Liu, and X. Yuan, "Hyperspectral image classification using spectral-spatial LSTMs," *Neurocomputing*, vol. 328, pp. 39–47, Feb. 2019.

[11] R. Hang, Q. Liu, D. Hong, and P. Ghamisi, "Cascaded recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5384–5394, Aug. 2019.

[12] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jul. 2015.

[13] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[14] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[15] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Feb. 2019.

[16] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[17] M. Oquab, L. Bottou, I. Laptev, and J. Sivic, "Learning and transferring mid-level image representations using convolutional neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 1717–1724.

[18] Q. Liu, R. Hang, H. Song, and Z. Li, "Learning multiscale deep features for high-resolution satellite image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 1, pp. 117–126, Jan. 2018.

[19] Y. Kong, X. Wang, and Y. Cheng, "Spectral–spatial feature extraction for HSI classification based on supervised hypergraph and sample expanded CNN," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 11, pp. 4128–4140, Nov. 2018.

[20] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 844–853, Feb. 2017.

[21] L. Jiao, M. Liang, H. Chen, S. Yang, H. Liu, and X. Cao, "Deep fully convolutional network-based spatial distribution prediction for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 10, pp. 5585–5599, Oct. 2017.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.

[22] G. Cheng, Z. Li, J. Han, X. Yao, and L. Guo, "Exploring hierarchical convolutional features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 11, pp. 6712–6722, Nov. 2018.
[23] Y. Li, W. Xie, and H. Li, "Hyperspectral image reconstruction by deep convolutional neural network for classification," *Pattern Recognit.*, vol. 63, pp. 371–383, Mar. 2017.
[24] L. Mou, P. Ghamisi, and X. X. Zhu, "Unsupervised spectral–spatial feature learning via deep residual Conv–Deconv network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 1, pp. 391–406, Jan. 2018.
[25] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
[26] M. Mirza and S. Osindero, "Conditional generative adversarial nets," 2014, *arXiv:1411.1784*. [Online]. Available: http://arxiv.org/abs/1411.1784
[27] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," 2015, *arXiv:1511.06434*. [Online]. Available: http://arxiv.org/abs/1511.06434
[28] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2223–2232.
[29] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley, "Least squares generative adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2794–2802.
[30] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 214–223.
[31] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1125–1134.
[32] K. Enomoto *et al.*, "Filmy cloud removal on satellite imagery with multispectral conditional generative adversarial nets," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jul. 2017, pp. 48–56.
[33] P. Ghamisi and N. Yokoya, "IMG2DSM: Height simulation from single imagery using conditional generative adversarial net," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 5, pp. 794–798, May 2018.
[34] S. Xu, X. Mu, D. Chai, and X. Zhang, "Remote sensing image scene classification based on generative adversarial networks," *Remote Sens. Lett.*, vol. 9, no. 7, pp. 617–626, Jul. 2018.
[35] Z. He, H. Liu, Y. Wang, and J. Hu, "Generative adversarial networks-based semi-supervised learning for hyperspectral image classification," *Remote Sens.*, vol. 9, no. 10, p. 1042, Oct. 2017.
[36] Y. Zhan, D. Hu, Y. Wang, and X. Yu, "Semisupervised hyperspectral image classification based on generative adversarial networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 212–216, Feb. 2018.
[37] M. Zhang, M. Gong, Y. Mao, J. Li, and Y. Wu, "Unsupervised feature extraction in hyperspectral images based on Wasserstein generative adversarial network," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 5, pp. 2669–2688, May 2019.
[38] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5046–5063, Sep. 2018.
[39] W. Zhao, X. Chen, Y. Bo, and J. Chen, "Semisupervised hyperspectral image classification with cluster-based conditional generative adversarial net," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 3, pp. 539–543, Mar. 2020.
[40] Y. Xu, L. Zhang, B. Du, and F. Zhang, "Spectral–spatial unified networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 10, pp. 5893–5909, Oct. 2018.
[41] C. Ledig *et al.*, "Photo-realistic single image super-resolution using a generative adversarial network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4681–4690.
[42] C. Debes *et al.*, "Hyperspectral and LiDAR data fusion: Outcome of the 2013 GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2405–2418, Jun. 2014.
[43] Y. Xu *et al.*, "Advanced multi-sensor optical remote sensing for urban land use and land cover classification: Outcome of the 2018 IEEE GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 6, pp. 1709–1724, Jun. 2019.
[44] B. Rasti *et al.*, "Feature extraction for hyperspectral imagery: The evolution from shallow to deep (overview and toolbox)," *IEEE Geosci. Remote Sens. Mag.*, early access, Apr. 29, 2020, doi: 10.1109/MGRS.2020.2979764.



Renlong Hang (Member, IEEE) received the M.S. and Ph.D. degrees from the Nanjing University of Information Science and Technology, Nanjing, China, in 2014 and 2017, respectively.

Since 2017, he has been a Lecturer with the School of Automation, Nanjing University of Information Science and Technology. From 2018 to 2019, he was a Post-Doctoral Researcher with the Department of Computer Science and Electrical Engineering, University of Missouri-Kansas City, Kansas City, MO, USA. He has authored or coauthored over 20 peer-reviewed articles in international journals, such as the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, the IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING, and the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS. His research interests include machine learning, pattern recognition, and their applications to remote sensing image processing.



Feng Zhou received the B.S. degree in electrical engineering and automation from the Nanjing University of Information Science and Technology, Nanjing, China, in 2016, where he is pursuing the Ph.D. degree in information and communication engineering with the School of Automation.

His research interests include deep learning and pattern recognition.



Qingshan Liu (Senior Member, IEEE) received the M.S. degree from Southeast University, Nanjing, China, in 2000, and the Ph.D. degree from the Chinese Academy of Sciences, Beijing, China, in 2003.

From 2010 to 2011, he was an Assistant Research Professor with Computational Biomedicine Imaging and Modeling Center, Department of Computer Science, Rutgers, The State University of New Jersey, Piscataway, NJ, USA. Before he joined Rutgers University, he was an Associate Professor with the National Laboratory of Pattern Recognition, Chinese Academy of Sciences. From June 2004 to April 2005, he was an Associate Researcher with Multimedia Laboratory, The Chinese University of Hong Kong, Hong Kong. He is a Professor with the School of Information and Control, Nanjing University of Information Science and Technology, Nanjing. His research interest includes image and vision analysis.

Dr. Liu received the President Scholarship of the Chinese Academy of Sciences in 2003.



Pedram Ghamisi (Senior Member, IEEE) received the B.Sc. degree in civil (survey) engineering from the Tehran South Campus of Azad University, Tehran, Iran, the M.Sc. degree (Hons.) in remote sensing from the K. N. Toosi University of Technology, Tehran, in 2012, and the Ph.D. degree in electrical and computer engineering from the University of Iceland, Reykjavik, Iceland, in 2015.

In 2013/2014, he spent seven months at the School of Geography, Planning and Environmental Management, The University of Queensland, Brisbane, QLD, Australia. After Ph.D., he subsequently worked as a Post-Doctoral Research Fellow at the University of Iceland. In 2015, he won the prestigious "Alexander von Humboldt Fellowship" and started his work as a Post-Doctoral Research Fellow at the Technical University of Munich (TUM), Munich, Germany, and Heidelberg University, Heidelberg, Germany, since October 2015. He was also a Research Scientist at German Aerospace Center (DLR), Remote Sensing Technology Institute (IMF), Oberpfaffenhofen, Germany, from October 2015 to May 2018. In 2018, he won the prestigious "High Potential Program" and started his work as Head of Machine Learning Group, Helmholtz-Zentrum Dresden-Rossendorf (HZDR), Freiberg, Germany. His research interests involve interdisciplinary research on remote sensing and machine (deep) learning, image and signal processing, and multisensory data fusion.

Dr. Ghamisi serves as an Associate Editor for the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS (GRSL) and *Remote Sensing*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 03:47:35 UTC from IEEE Xplore. Restrictions apply.